BeatAhead Phase 11: Production Deployment Hardening & Architectural Audit Report

Status: Audit Complete — Production Deployment HELD (Blocker Identified)
Date: September 2026
Repositories: `C:\ischemic` (Branch: `staging-ml-integration`) & `c:\ML Model - Ischemic`
Model Dependency: Frozen Phase 5 XGBoost Matrix A (`models/beatahead_phase5_model.joblib`)
Operating Threshold: `0.156742` (Immutable, Derived from Validation Fold F_1 Maximization)
Schema: 26-Feature Matrix A (`models/feature_schema.json`)
Automated Regression Pass Rate: 53 / 53 Tests Passing (100%)
Production Deployment Decision: **BLOCKED (Vercel Serverless ML Architecture Incompatibility)**

Executive Summary

Phase 11 performed a formal production deployment hardening, environmental security audit, and runtime architecture evaluation of the BeatAhead platform. The objective was to ascertain whether the current staging integration—which successfully pairs the Next.js web application (`C:\ischemic`) with the frozen Python machine learning inference service—is operationally viable for direct serverless deployment to Vercel.

Key Finding: Deployment Blocker Identified

A **critical architectural blocker** was identified: The Next.js staging integration in `src/app/api/ml/predict/route.ts` relies on connecting to a persistent local Python daemon (`http://127.0.0.1:8000`) and falls back to a synchronous child process invocation (`execFileSync`) pointing to local Windows directory paths (`C:\ML Model - Ischemic\.venv\Scripts\python.exe`).

In a standard Vercel production deployment: 1. **No Persistent Daemon**: Vercel executes Next.js API routes inside ephemeral, stateless Linux serverless microVMs; `127.0.0.1:8000` does not exist and connections will be immediately refused (`ECONNREFUSED`). 2. **Local Path Invalidation**: The fallback paths (`C:\ML Model - Ischemic\...`) do not exist in the Linux container filesystem (`/var/task/`). 3. **Missing Python ML Environment**: The standard Vercel Node.js runtime does not bundle Python, XGBoost, Scikit-learn, Joblib, or NumPy.

Therefore, **direct deployment to Vercel (`vercel --prod`) will fail at the inference boundary**. In accordance with Phase 11 instructions, this blocker is explicitly recorded and production deployment is strictly **HALTED**.

---

1. Deployment Architecture Audit (Task 1)

Current Staging Data Flow



Failure Mechanism on Vercel Production

When deployed to Vercel without external service provisioning: - **`fetch("http://127.0.0.1:8000/predict")`**: Times out or throws `fetch failed (ECONNREFUSED)`. - **`execFileSync("C:\\ML Model - Ischemic...")`**: Throws `ENOENT` (path not found) or `EACCES` on Linux. - **Frontend**

User Experience: Returns `503 Service Unavailable` with sanitized error code `ML_SERVICE_ERROR`. The web dashboard safely suppresses predictions, but live inference becomes non-functional.

2. Production ML Architecture Options Evaluation (Task 2)

Four production architectural options were evaluated:

Architecture Option	Description	Technical Feasibility	Model Invariance	Verdict
Option A: Dedicated Python Container	Deploy `src/ml_service.py` to a dedicated container runtime (e.g., AWS ECS, Google Cloud Run, Fly.io, Railway, or AWS Lambda with Container Image). Next.js on Vercel connects via `ML_SERVICE_URL=https://ml.beatahead.internal/predict`.	**High**	100% Unchanged (`beatahead_phase5_model.joblib`)	**RECOMMENDED FOR PRODUCTION**
Option B: Vercel Python Serverless Function	Reconfigure the repo to expose a native Vercel Python Serverless Function (`api/predict.py`). Bundles Python requirements (`requirements.txt`).	**Medium / High Risk**	100% Unchanged	**WARNING**: Heavy ML dependencies (`scikit-learn`, `xgboost`, `scipy`, `numpy`) often exceed Vercel's 250 MB unzipped function size limit.
Option C: Runtime Export (ONNX / Treelite)	Convert frozen XGBoost tree ensemble into ONNX or compile to native C/WASM/Node.js library for in-process execution without Python.	**Technical Work Required**	Requires formal numerical equivalence audit	**HOLD**: Phase 11 prohibits model conversion without explicit authorization.
Option D: Staging-Only ML Boundary	Keep Python ML inference strictly in local staging/simulation; deploy website with simulated physiological telemetry.	**Immediate**	100% Unchanged	**VIABLE FOR DEMO ONLY**: Does not support live hardware inference in production.

3. Model Artifact Security Audit (Task 3)

The project filesystem and compilation artifacts were inspected to prevent intellectual property exposure or data leakage:

Security Check	Verification Method	Observed Result	Status
Public Folder Isolation	Searched `C:\ischemic\public\` for `*.joblib`, `*.pkl`, `*.pt`	0 Model artifacts found	**PASS**
Client Bundle Isolation	Searched `C:\ischemic\.next\` static chunks for model binaries	0 Model artifacts bundled	**PASS**
Direct Download Prevention	Attempted HTTP GET for model binary paths	Returns 404 Not Found	**PASS**
Filesystem Path Suppression	Tested error outputs across all API failure modes	Zero paths (`C:\...`) returned in HTTP responses	**PASS**
Python Stack Trace Suppression	Tested malformed, out-of-bounds, and crashing payloads	Generic error messages returned; zero tracebacks exposed	**PASS**
Metadata Protection	Inspected `models/model_metadata.json`	No secrets, credentials, or private paths stored	**PASS**

4. Environment Variables Audit (Task 4)

An audit of `C:\ischemic\.env.example` and `C:\ischemic\.env.local` was conducted to verify that no server secrets are exposed to browser clients:

Client-Accessible Variables (`NEXT_PUBLIC_*`)

- `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` : Public authentication key (Required for Clerk frontend SDK).
- `NEXT_PUBLIC_VOICE_SERVER_WS_URL` : WebSocket endpoint for streaming voice assistance.
- **Audit Result**: Zero secrets, database passwords, or private API keys are exposed via `NEXT_PUBLIC_*` .

Server-Only Secrets (Protected from Browser Client)

- `CLERK_SECRET_KEY` : Clerk backend authentication secret.
- `GEMINI_API_KEY` : Google Gemini API key.
- `GROQ_API_KEY` : Groq API key for low-latency STT.
- `ELEVENLABS_API_KEY` : ElevenLabs TTS synthesis key.
- `EXOTEL_API_KEY` & `EXOTEL_API_TOKEN` : Telephony credentials.
- **Audit Result**: All backend secrets are strictly restricted to Node.js server execution contexts.

---

5. API Security & Input Validation (Task 5)

The `POST /api/ml/predict` endpoint implements strict input sanitization: 1. **Payload Type Enforcement**: Rejects non-JSON payloads, arrays, and malformed strings with `400 Bad Request` . 2. **Schema Completeness**: Validates that all 26 Matrix A features are present. Missing keys are rejected immediately with `missing_features` details. 3. **Strict Bounds Checking**: Every numerical feature is compared against physiologically plausible human ranges (`FEATURE_BOUNDS`). Impossible heart rates (e.g. 450 bpm) or corrupted voltages are rejected with `400 Bad Request` . 4. **Non-Finite Value Rejection**: `NaN` , `+Infinity` , and `-Infinity` are rejected before reaching the inference model. 5. **Timeout Protection**: Upstream network requests to `ML_SERVICE_URL` enforce an explicit 2,000 ms timeout via `AbortController` , preventing worker thread exhaustion.

---

6. Failure Modes & Graceful Degradation (Task 6)

The staging platform enforces fail-safe degradation across all system failure states:

Failure State	Root Cause	System Response	Stale Output Displayed?
ML Service Unavailable	Daemon stopped / container offline	Returns `503 Service Unavailable` ; UI displays `Inference Service Offline` banner.	**NEVER**
Model Initialization Failure	Missing artifact / permission error	Returns `503 ML_SERVICE_ERROR` ; logs to server console.	**NEVER**
Sensor Disconnection	Missing PPG / ECG streams	$Q_{\text{overall}} < 0.35$; triggers `Insufficient Signal Quality` state.	**NEVER**
High Motion Artifact	Patient walking / running ($M > 0.70$)	Suppresses false alarms; prompts user to stabilize sensor contact.	**NEVER**
Baseline Establishing	Monitoring time $< 600\text{ s}$	Holds state in `Baseline Establishing` ; inhibits premature stable classification.	**NEVER**

---

7. Production Health Check Endpoint (Task 7)

A dedicated, non-sensitive health check route was created at `C:\ischemic\src\app\api\ml\health\route.ts` .

Endpoint: `GET /api/ml/health`

Sample Response (JSON):



Security Characteristics: - No filesystem paths (`C:\...` or `/var/...`) are exposed. - No server usernames or process IDs are returned. - If the upstream Python daemon is unreachable, the endpoint cleanly reports `"status": "degraded"` with `"upstream_service.status": "unreachable"` without crashing.

8. Deployment Configuration Audit (Task 8)

Configuration Item	File Path	Current Value	Production Impact
Node.js Target	`package.json`	Next.js 15.1.0, React 19.0.0	Compatible with Vercel Node 20.x runtime.
Build Command	`package.json`	`next build`	Compiles cleanly across 38 routes in 9.0s.
TypeScript Config	`tsconfig.json`	Strict mode enabled	0 Errors across codebase (`tsc --noEmit`).
Hardcoded Windows Paths	`src/app/api/ml/predict/route.ts`	`C:\\ML Model - Ischemic`	**DEPLOYMENT BLOCKER** on Linux serverless.
Localhost Coupling	`src/app/api/ml/predict/route.ts`	`http://127.0.0.1:8000`	**DEPLOYMENT BLOCKER** without external host.

9. Performance Latency Profile (Task 9)

In accordance with Phase 11 guidelines, cold-start initialization is strictly distinguished from warm in-process inference latency:

Operational Phase	Measured Latency	Explanation
Cold-Start / Initialization	**1,544.24 ms**	One-time overhead: Python runtime startup, Joblib file reading, deserialization of 200 XGBoost decision trees, and JSON schema parsing.
Warm In-Process Inference (Mean)	**5.314 ms**	Real-time vectorized tree evaluation across 26 features.
Warm In-Process Inference (P50)	**4.456 ms**	Median evaluation time per single-window query.
Warm In-Process Inference (P95)	**6.953 ms**	95th percentile worst-case evaluation time under concurrent load.
Memory Allocation	**0.06 MB**	Peak temporary memory allocated per inference call.

10. Safety & Regulatory Copy Audit (Task 10)

All production-facing UI pages were verified against medical device disclaimer requirements: - **Disclaimer Visibility**: All primary dashboards display the standard notice:

> *"Research prototype screening system — not a medical diagnostic device. For investigational demonstration only."* - **Prohibited Terminology**: An automated scan confirmed **0 occurrences** of diagnostic assertions: - `"heart attack detected"` : **0 occurrences** - `"ischemia confirmed"` : **0 occurrences** - `"clinically proven"` : **0 occurrences** - `"diagnosis"` : **0 occurrences in model outputs** - **Simulation Mode**: The `SimulatedBadge` prominently flags simulated telemetry with `RESEARCH MODEL — SIMULATED INPUT` .

11. Deployment Checklist & Classification Matrix (Task 11)

Audit Item	Area	Evaluation Result	Classification
Model Binary Immutability	ML Core	SHA-256 hash verified; zero retraining or alteration	**PASS**
Decision Threshold Freeze	ML Core	Threshold fixed at `0.156742`	**PASS**
Automated Regression Suite	Quality Assurance	53 / 53 Tests Passing	**PASS**
TypeScript Static Check	Frontend	0 Compilation Errors	**PASS**
Next.js Production Build	Frontend	38 Routes compiled successfully	**PASS**
Model Artifact Leakage	Security	0 `.joblib` files in public or client bundles	**PASS**
Environment Variable Isolation	Security	Zero secrets in `NEXT_PUBLIC_*`	**PASS**
Safety Disclaimers & Copy	Compliance	Non-diagnostic language enforced; disclaimers active	**PASS**
Health Check Route	Operations	`GET /api/ml/health` operational	**PASS**
External ML Service Host	Infrastructure	Upstream service URL not yet provisioned in cloud	**WARNING**
Vercel Serverless Python Host	Architecture	Vercel Node.js runtime cannot run local Python daemon	**BLOCKER**
Hardcoded Windows Fallback	Architecture	Local `C:\...` paths in `route.ts` will fail on Linux	**BLOCKER**

12. Deployment Decision (Task 12)

> [!CAUTION] **PRODUCTION DEPLOYMENT HALTED — BLOCKERS IDENTIFIED**
>
> In strict compliance with Phase 11 directives: - `vercel --prod` was **NOT** executed. - No production DNS or infrastructure changes were initiated.
>
> **Prerequisites for Future Production Release**: 1. Deploy `src/ml_service.py` to a dedicated containerized cloud service (e.g. AWS ECS, Google Cloud Run, or Railway) configured with the frozen `beatahead_phase5_model.joblib` . 2. Set `ML_SERVICE_URL` in Vercel environment variables to point to the secure HTTPS cloud container endpoint. 3. Remove the local Windows `execFileSync` fallback from `route.ts` before triggering the production Vercel build.